

Cryptography and Network Security Chapter 9

Fifth Edition
by William Stallings

Lecture slides by Lawrie Brown

Lecture slides by Lawrie Brown for “Cryptography and Network Security”, 5/e, by William Stallings, Chapter 9 – “Public Key Cryptography and RSA”.

Chapter 9 – Public Key Cryptography and RSA

Every Egyptian received two names, which were known respectively as the true name and the good name, or the great name and the little name; and while the good or little name was made public, the true or great name appears to have been carefully concealed.

—*The Golden Bough*, Sir James George Frazer

Opening quote.

Private-Key Cryptography

- traditional **private/secret/single key** cryptography uses **one** key
- shared by both sender and receiver
- if this key is disclosed communications are compromised
- also is **symmetric**, parties are equal
- hence does not protect sender from receiver forging a message & claiming is sent by sender

The development of public-key cryptography is the greatest and perhaps the only true revolution in the entire history of cryptography. From its earliest beginnings to modern times, virtually all cryptographic systems have been based on the elementary tools of substitution and permutation, and can be classed as private/secret/single key (symmetric) systems. All classical, and modern block and stream ciphers are of this form.

Public-Key Cryptography

- probably most significant advance in the 3000 year history of cryptography
- uses **two** keys – a public & a private key
- **asymmetric** since parties are **not** equal
- uses clever application of number theoretic concepts to function
- complements **rather than** replaces private key crypto

Will now discuss the radically different **public key** systems, in which **two keys** are used. Public-key cryptography provides a radical departure from all that has gone before. The development of public-key cryptography is the greatest and perhaps the only true revolution in the entire history of cryptography. It is asymmetric, involving the use of two separate keys, in contrast to symmetric encryption, that uses only one key. Anyone knowing the public key can encrypt messages or verify signatures, but **cannot** decrypt messages or create signatures, counter-intuitive though this may seem. The use of two keys has profound consequences in the areas of confidentiality, key distribution, and authentication. It works by the clever use of number theory problems that are easy one way but hard the other. Note that public key schemes are neither more nor less secure than private key (security depends on the key size for both), nor do they replace private key schemes (they are too slow to do so), rather they complement them. Both also have issues with key distribution, requiring the use of some suitable protocol.

Why Public-Key Cryptography?

- developed to address two key issues:
 - **key distribution** – how to have secure communications in general without having to trust a KDC with your key
 - **digital signatures** – how to verify a message comes intact from the claimed sender
- public invention due to Whitfield Diffie & Martin Hellman at Stanford Uni in 1976
 - known earlier in classified community

The concept of public-key cryptography evolved from an attempt to attack two of the most difficult problems associated with symmetric encryption: key distribution and digital signatures. The first problem is that of key distribution, which under symmetric encryption requires either (1) that two communicants already share a key, which somehow has been distributed to them; or (2) the use of a key distribution center. This seemed to negate the very essence of cryptography: the ability to maintain total secrecy over your own communication. The second was that of "digital signatures." If the use of cryptography was to become widespread, not just in military situations but for commercial and private purposes, then electronic messages and documents would need the equivalent of signatures used in paper documents.

The idea of public key schemes, and the first practical scheme, which was for key distribution only, was published in 1976 by Diffie & Hellman. The concept had been previously described in a classified report in 1970 by James Ellis (UK CESG) - and subsequently declassified [ELLI99]. Its interesting to note that they discovered RSA first, then Diffie-Hellman, opposite to the order of public discovery! There is also a claim that the NSA knew of the concept in the mid-60's [SIMM93].

Public-Key Cryptography

- **public-key/two-key/asymmetric** cryptography involves the use of **two** keys:
 - a **public-key**, which may be known by anybody, and can be used to **encrypt messages**, and **verify signatures**
 - a related **private-key**, known only to the recipient, used to **decrypt messages**, and **sign (create) signatures**
- **infeasible to determine private key from public**
- **is asymmetric** because
 - those who encrypt messages or verify signatures **cannot** decrypt messages or create signatures

Asymmetric algorithms rely on one key for encryption and a different but related key for decryption. These algorithms have the following important characteristic:

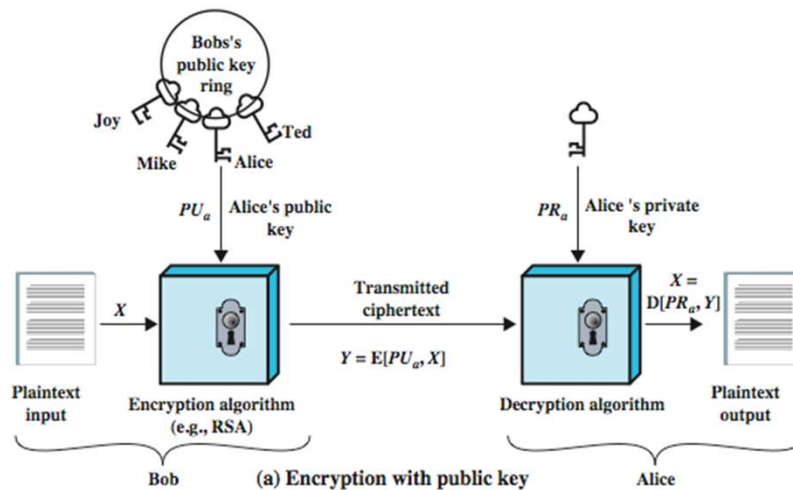
- It is computationally infeasible to determine the decryption key given only knowledge of the cryptographic algorithm and the encryption key.

In addition, some algorithms, such as RSA, also exhibit the following characteristic:

- Either of the two related keys can be used for encryption, with the other used for decryption.

Anyone knowing the public key can encrypt messages or verify signatures, but **cannot** decrypt messages or create signatures, thanks to some clever use of number theory.

Public-Key Cryptography



Stallings Figure 9.1a “Public-Key Cryptography”, shows that a public-key encryption scheme has six ingredients:

- Plaintext: the readable message /data fed into the algorithm as input.
- Encryption algorithm: performs various transformations on the plaintext.
- Public and private keys: a pair of keys selected so that if one is used for encryption, the other is used for decryption. The exact transformations performed by the algorithm depend on the public or private key that is provided as input.
- Ciphertext: the scrambled message produced as output. It depends on the plaintext and the key. For a given message, two different keys will produce two different ciphertexts.
- Decryption algorithm: accepts the ciphertext and matching key and produces the original plaintext.

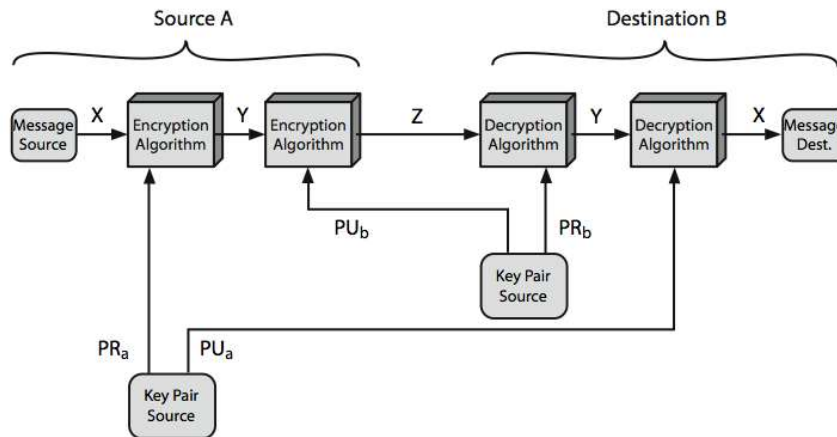
Consider the following analogy using padlocked boxes: traditional schemes involve the sender putting a message in a box and locking it, sending that to the receiver, and somehow securely also sending them the key to unlock the box. The radical advance in public key schemes was to turn this around, the receiver sends an **unlocked box** (their public key) to the sender, who puts the message in the box and locks it (easy - and having locked it cannot get at the message), and sends the locked box to the receiver who can unlock it (also easy), having the (private) key. An attacker would have to pick the lock on the box (hard).

Symmetric vs Public-Key

Conventional Encryption	Public-Key Encryption
<i>Needed to Work:</i> 1. The same algorithm with the same key is used for encryption and decryption. 2. The sender and receiver must share the algorithm and the key. <i>Needed for Security:</i> 1. The key must be kept secret. 2. It must be impossible or at least impractical to decipher a message if no other information is available. 3. Knowledge of the algorithm plus samples of ciphertext must be insufficient to determine the key.	<i>Needed to Work:</i> 1. One algorithm is used for encryption and decryption with a pair of keys, one for encryption and one for decryption. 2. The sender and receiver must each have one of the matched pair of keys (not the same one). <i>Needed for Security:</i> 1. One of the two keys must be kept secret. 2. It must be impossible or at least impractical to decipher a message if no other information is available. 3. Knowledge of the algorithm plus one of the keys plus samples of ciphertext must be insufficient to determine the other key.

Stallings Table 9.2 summarizes some of the important aspects of symmetric and public-key encryption. To discriminate between the two, we refer to the key used in symmetric encryption as a **secret key**. The two keys used for asymmetric encryption are referred to as the **public key** and the **private key**. Invariably, the private key is kept secret, but it is referred to as a private key rather than a secret key to avoid confusion with symmetric encryption.

Public-Key Cryptosystems



Stallings Figure 9.4 “Public-Key Cryptosystems: Secrecy and Authentication” illustrates the essential elements of a public-key encryption scheme.

Note that public-key schemes can be used for either secrecy or authentication, or both (as shown here). There is some source A that produces a message in plaintext X. The elements of X are letters in some finite alphabet. The message is intended for destination B. B generates a related pair of keys: a public key, PU_b , and a private key, PR_b . PR_b is known only to B, whereas PU_b is publicly available and therefore accessible by A. With the message X and the encryption key PU_b as input, A forms the ciphertext $Y = E(PU_b, X)$. The intended receiver, in possession of the matching private key, is able to invert the transformation: $X = D(PR_b, Y)$. An adversary, observing Y and having access to PU_b , but not having access to PR_b or X, must attempt to recover X and/or PR_b . This provides confidentiality. Can also use a public-key encryption to provide authentication: $Y = E(PR_a, X)$; $X = D(PU_a, Y)$. To provide both the authentication function and confidentiality have a double use of the public-key scheme (as shown here): $Z = E(PU_b, E(PR_a, X))$; $X = D(PU_a, D(PR_b, Z))$. In this case, separate key pairs are used for each of these purposes. The receiver owns and creates secrecy keys, sender owns and creates authentication keys.

In practice typically DO NOT do this, because of the computational cost of public-key schemes. Rather encrypt a session key which is then used with a block cipher to encrypt the actual message, and separately sign a hash of the message as a digital signature - this will be discussed more later.

Public-Key Applications

- can classify uses into 3 categories:
 - **encryption/decryption** (provide secrecy)
 - **digital signatures** (provide authentication)
 - **key exchange** (of session keys)
- some algorithms are suitable for all uses, others are specific to one

Algorithm	Encryption/Decryption	Digital Signature	Key Exchange
RSA	Yes	Yes	Yes
Elliptic Curve	Yes	Yes	Yes
Diffie-Hellman	No	No	Yes
DSS	No	Yes	No

Public-key systems are characterized by the use of a cryptographic type of algorithm with two keys. Depending on the application, the sender uses either the sender's private key or the receiver's public key, or both, to perform some type of cryptographic function. In broad terms, we can classify the use of public-key cryptosystems into the three categories:

- **Encryption/decryption:** The sender encrypts a message with the recipient's public key.
- **Digital signature:** The sender "signs" a message with its private key, either to the whole message or to a small block of data that is a function of the message.
- **Key exchange:** Two sides cooperate to exchange a session key. Several different approaches are possible, involving the private key(s) of one or both parties.

Some algorithms are suitable for all three applications, whereas others can be used only for one or two of these applications. Stallings Table 9.3 (shown here) indicates the applications supported by the algorithms discussed in this book.

Public-Key Requirements

- Public-Key algorithms rely on two keys where:
 - it is computationally infeasible to find decryption key knowing only algorithm & encryption key
 - it is computationally easy to en/decrypt messages when the relevant (en/decrypt) key is known
 - either of the two related keys can be used for encryption, with the other used for decryption (for some algorithms)
- these are formidable requirements which only a few algorithms have satisfied

The cryptosystem illustrated in Figures 9.2 through 9.4 depends on a cryptographic algorithm based on two related keys. Diffie and Hellman postulated this system without demonstrating that such algorithms exist. However, they did lay out the conditions that such algorithms must fulfill:

1. It is computationally easy for a party B to generate a pair (public key PUB , private key PRb).
2. It is computationally easy for a sender A, knowing the public key and the message to be encrypted, M , to generate the corresponding ciphertext: $C = E(PUB, M)$
3. It is computationally easy for the receiver B to decrypt the resulting ciphertext using the private key to recover the original message: $M = D(PRb, C) = D[PRb, E(PUB, M)]$
4. It is computationally infeasible for an adversary, knowing the public key, Pb , to determine the private key, PRb
5. It is computationally infeasible for an adversary, knowing the public key, Pb , and a ciphertext, C , to recover the original message, M .
6. (optional) The two keys can be applied in either order:

$$M = D[PU, E(PR, M)] = D[PR, E(PU, M)]$$

These are formidable requirements, as evidenced by the fact that only a few algorithms (RSA, elliptic curve cryptography, Diffie-Hellman, DSS) have received widespread acceptance in the several decades since the concept of public-key cryptography was proposed.

Public-Key Requirements

- need a trapdoor one-way function
- one-way function has
 - $Y = f(X)$ easy
 - $X = f^{-1}(Y)$ infeasible
- a trap-door one-way function has
 - $Y = f_k(X)$ easy, if k and X are known
 - $X = f_k^{-1}(Y)$ easy, if k and Y are known
 - $X = f_k^{-1}(Y)$ infeasible, if Y known but k not known
- a practical public-key scheme depends on a suitable trap-door one-way function

The requirements boil down to the need for a trap-door one-way function. A one-way function is one that maps a domain into a range such that every function value has a unique inverse, with the condition that the calculation of the function is easy whereas the calculation of the inverse is infeasible:

$Y = f(X)$ easy

$X = f^{-1}(Y)$ infeasible

Generally, *easy* is defined to mean a problem that can be solved in polynomial time as a function of input length. The term *infeasible* is a much fuzzier concept. In general, we can say a problem

Now consider a trap-door one-way function, which is easy to calculate in one direction and infeasible to calculate in the other direction unless certain additional information is known. With the additional information the inverse can be calculated in polynomial time. We can summarize as follows: A trap-door one-way function is a family of invertible functions f_k , such that:

$Y = f_k(X)$ easy, if k and X are known

$X = f_k^{-1}(Y)$ easy, if k and Y are known

$X = f_k^{-1}(Y)$ infeasible, if Y known but k not known

Thus, the development of a practical public-key scheme depends on discovery of a suitable trap-door one-way function.

Security of Public Key Schemes

- like private key schemes brute force **exhaustive search** attack is always theoretically possible
- but keys used are too large (>512bits)
- security relies on a **large enough** difference in difficulty between **easy** (en/decrypt) and **hard** (cryptanalyse) problems
- more generally the **hard** problem is known, but is made hard enough to be impractical to break
- requires the use of **very large numbers**
- hence is **slow** compared to private key schemes

Public key schemes are no more or less secure than private key schemes - in both cases the size of the key determines the security. As with symmetric encryption, a public-key encryption scheme is vulnerable to a brute-force attack. The countermeasure is the same: Use large keys. However, there is a tradeoff to be considered. Public-key systems depend on the use of some sort of invertible mathematical function. The complexity of calculating these functions may not scale linearly with the number of bits in the key but grow more rapidly than that. Thus, the key size must be large enough to make brute-force attack impractical but small enough for practical encryption and decryption. In practice, the key sizes that have been proposed do make brute-force attack impractical but result in encryption/decryption speeds that are too slow for general-purpose use. Instead, as was mentioned earlier, public-key encryption is currently confined to key management and signature applications. Another form of attack is to find some way to compute the private key given the public key. To date, it has not been mathematically proven that this form of attack is infeasible for a particular public-key algorithm.

Note also that you can't compare key sizes - a 64-bit private key scheme has very roughly similar security to a 512-bit RSA - both could be broken given sufficient resources. But with public key schemes at least there is usually a firmer theoretical basis for determining the security since its based on well-known and well studied number theory problems.

RSA

- by Rivest, Shamir & Adleman of MIT in 1977
- best known & widely used public-key scheme
- based on exponentiation in a finite (Galois) field over integers modulo a prime
 - nb. exponentiation takes $O((\log n)^3)$ operations (easy)
- uses large integers (eg. 1024 bits)
- security due to cost of factoring large numbers
 - nb. factorization takes $O(e^{\log n \log \log n})$ operations (hard)

RSA is the best known, and by far the most widely used general public key encryption algorithm, and was first published by Rivest, Shamir & Adleman of MIT in 1978 [RIVE78]. The Rivest-Shamir-Adleman (RSA) scheme has since that time reigned supreme as the most widely accepted and implemented general-purpose approach to public-key encryption. It is based on exponentiation in a finite (Galois) field over integers modulo a prime, using large integers (eg. 1024 bits). Its security is due to the cost of factoring large numbers.

RSA En/decryption

- to encrypt a message M the sender:
 - obtains **public key** of recipient $PU = \{e, n\}$
 - computes: $C = M^e \bmod n$, where $0 \leq M < n$
- to decrypt the ciphertext C the owner:
 - uses their private key $PR = \{d, n\}$
 - computes: $M = C^d \bmod n$
- note that the message M must be smaller than the modulus n (block if needed)

The scheme developed by Rivest, Shamir, and Adleman makes use of an expression with exponentials. Plaintext is encrypted in blocks, with each block having a binary value less than some number n . The actual RSA encryption and decryption computations are each simply a single exponentiation mod (n) . Both sender and receiver must know the value of n . The sender knows the value of e , and only the receiver knows the value of d . Thus, this is a public-key encryption algorithm with a public key of $PU = \{e, n\}$ and a private key of $PR = \{d, n\}$. Note that the message must be smaller than the modulus. The “magic” is in the choice of the modulus and exponents which makes the system work.

RSA Key Setup

- each user generates a public/private key pair by:
- selecting two large primes at random: p, q
- computing their system modulus $n=p \cdot q$
 - note $\phi(n) = (p-1)(q-1)$
- selecting at random the encryption key e
 - where $1 < e < \phi(n)$, $\gcd(e, \phi(n)) = 1$
- solve following equation to find decryption key d
 - $e \cdot d = 1 \pmod{\phi(n)}$ and $0 \leq d \leq n$
- publish their public encryption key: $PU=\{e,n\}$
- keep secret private decryption key: $PR=\{d,n\}$

The required modulus and exponent values are chosen during key setup. RSA key setup is done once (rarely) when a user establishes (or replaces) their public key, using the steps as shown. The exponent e is usually fairly small, just must be relatively prime to $\phi(n)$. Need to compute its inverse mod $\phi(n)$ to find d . It is critically important that the factors p & q of the modulus n are kept secret, since if they become known, the system can be broken. Note that different users will have different moduli n .

Why RSA Works

- because of Euler's Theorem:
 - $a^{\phi(n)} \bmod n = 1$ where $\gcd(a, n) = 1$
- in RSA have:
 - $n = p \cdot q$
 - $\phi(n) = (p-1)(q-1)$
 - carefully chose e & d to be inverses mod $\phi(n)$
 - hence $e \cdot d = 1 + k \cdot \phi(n)$ for some k
- hence :

$$\begin{aligned}
 C^d &= M^{e \cdot d} = M^{1+k \cdot \phi(n)} = M^1 \cdot (M^{\phi(n)})^k \\
 &= M^1 \cdot (1)^k = M^1 = M \bmod n
 \end{aligned}$$

For this algorithm to be satisfactory for public-key encryption, it must be possible to find values of e , d , n such that $M^{ed} \bmod n = M$ for all $M < n$. We need to find a relationship of the form $M^{ed} \bmod n = M$. The preceding relationship holds if e and d are multiplicative inverses modulo $\phi(n)$, where $\phi(n)$ is the Euler totient function. This is a direct consequence of Euler's Theorem, so that raising a number to power e then d (or vica versa) results in the original number!

RSA Example - Key Setup

1. Select primes: $p=17$ & $q=11$
2. Calculate $n = pq = 17 \times 11 = 187$
3. Calculate $\phi(n) = (p-1)(q-1) = 16 \times 10 = 160$
4. Select e : $\gcd(e, 160) = 1$; **choose** $e=7$
5. Determine d : $de \equiv 1 \pmod{160}$ **and** $d < 160$
Value is $d=23$ since $23 \times 7 = 161 = 10 \times 160 + 1$
6. Publish public key $PU = \{7, 187\}$
7. Keep secret private key $PR = \{23, 187\}$

Stallings provides an example of RSA key generation using “trivial” sized numbers.

Selecting primes requires the use of a primality test.

Finding d as inverse of $e \pmod{\phi(n)}$ requires use of Euclid’s Inverse algorithm (see Ch4)

RSA Example - En/Decryption

- sample RSA encryption/decryption is:
- given message $M = 88$ (nb. $88 < 187$)
- encryption:

$$C = 88^7 \bmod 187 = 11$$

- decryption:

$$M = 11^{23} \bmod 187 = 88$$

Then show that the encryption and decryption operations are simple exponentiations mod 187.

Rather than having to laborious repeatedly multiply, can use the "square and multiply" algorithm with modulo reductions to implement all exponentiations quickly and efficiently (see next).

Exponentiation

- can use the Square and Multiply Algorithm
- a fast, efficient algorithm for exponentiation
- concept is based on repeatedly squaring base
- and multiplying in the ones that are needed to compute the result
- look at binary representation of exponent
- only takes $O(\log_2 n)$ multiples for number n
 - eg. $7^5 = 7^4 \cdot 7^1 = 3 \cdot 7 = 10 \pmod{11}$
 - eg. $3^{129} = 3^{128} \cdot 3^1 = 5 \cdot 3 = 4 \pmod{11}$

To perform the modular exponentiations, you can use the “Square and Multiply Algorithm”, a fast, efficient algorithm for doing exponentiation, which has a long history. The idea is to repeatedly square the base, and multiply in the ones that are needed to compute the result, as found by examining the binary representation of the exponent..

Exponentiation

```
c = 0; f = 1
for i = k downto 0
  do c = 2 x c
    f = (f x f) mod n
  if  $b_i == 1$  then
    c = c + 1
    f = (f x a) mod n
return f
```

State here one version of the “Square and Multiply Algorithm”, from Stallings Figure 9.8.

Efficient Encryption

- encryption uses exponentiation to power e
- hence if e small, this will be faster
 - often choose $e=65537$ ($2^{16}-1$)
 - also see choices of $e=3$ or $e=17$
- but if e too small (eg $e=3$) can attack
 - using Chinese remainder theorem & 3 messages with different moduli
- if e fixed must ensure $\gcd(e, \phi(n)) = 1$
 - ie reject any p or q not relatively prime to e

To speed up the operation of the RSA algorithm using the public key, can choose to use a small value of e . The most common choice is 65537 ($2^{16} + 1$); two other popular choices are 3 and 17. Each of these choices has only two 1 bits and so the number of multiplications required to perform exponentiation is minimized. However, with a very small public key, such as $e = 3$, RSA becomes vulnerable to a simple attack.

The reader may have noted that the definition of the RSA algorithm (Figure 9.5) requires that during key generation the user selects a value of e that is relatively prime to $\phi(n)$. Thus, if a value of e is selected first, and the primes p and q are generated, it may turn out that $\gcd(\phi(n), e) \neq 1$. In that case, the user must reject the p, q values and generate a new p, q pair.

Efficient Decryption

- decryption uses exponentiation to power d
 - this is likely large, insecure if not
- can use the Chinese Remainder Theorem (CRT) to compute mod p & q separately. then combine to get desired answer
 - approx 4 times faster than doing directly
- only owner of private key who knows values of p & q can use this technique

We cannot similarly choose a small constant value of d for efficient operation. A small value of d is vulnerable to a brute-force attack and to other forms of cryptanalysis [WIEN90]. However, there is a way to speed up computation using the Chinese Remainder Theorem (CRT) to compute mod p & q separately, and then combine results to get the desired answer, as shown in the text. This is approx 4 times faster than calculating " $C^d \bmod n$ " directly. Note that only the owner of the private key details (who knows the values of p & q) can do this, but of course that's exactly where help is needed, since if e is small then d will be likely be large!

RSA Key Generation

- users of RSA must:
 - determine two primes at random - p, q
 - select either e or d and compute the other
- primes p, q must not be easily derived from modulus $n=p \cdot q$
 - means must be sufficiently large
 - typically guess and use probabilistic test
- exponents e, d are inverses, so use Inverse algorithm to compute the other

Before the application of the public-key cryptosystem, each participant must generate a pair of keys, which requires finding primes and computing inverses. Both the prime generation and the derivation of a suitable pair of inverse exponents may involve trying a number of alternatives. Typically make random guesses for a possible p or q , and check using a probabilistic primality test whether the guessed number is indeed prime. If not, try again. Note that the prime number theorem shows that the average number of guesses needed is not too large. Then compute decryption exponent d using Euclid's Inverse Algorithm, which is quite efficient.

RSA Security

- possible approaches to attacking RSA are:
 - brute force key search - infeasible given size of numbers
 - mathematical attacks - based on difficulty of computing $\phi(n)$, by factoring modulus n
 - timing attacks - on running of decryption
 - chosen ciphertext attacks - given properties of RSA

Note some possible possible approaches to attacking the RSA algorithm, as shown.

The defense against the brute-force approach is the same for RSA as for other cryptosystems, namely, use a large key space. Thus the larger the number of bits in d , the better. However because the calculations involved both in key generation and in encryption/decryption are complex, the larger the size of the key, the slower the system will run.

Will now review the other possible types of attacks.

Factoring Problem

- mathematical approach takes 3 forms:
 - factor $n=p \cdot q$, hence compute $\phi(n)$ and then d
 - determine $\phi(n)$ directly and compute d
 - find d directly
- currently believe all equivalent to factoring
 - have seen slow improvements over the years
 - as of May-05 best is 200 decimal digits (663) bit with LS
 - biggest improvement comes from improved algorithm
 - cf QS to GNFS to LS
 - currently assume 1024-2048 bit RSA is secure
 - ensure p, q of similar size and matching other constraints

We can identify three approaches to attacking RSA mathematically, as shown. Mathematicians currently believe all equivalent to factoring.

See Stallings Table 9.4 (next slide) for progress in factoring, where see slow improvements over the years, with the biggest improvements coming from improved algorithms. The best current algorithm is the “Lattice Sieve” (LS), which replaced the “Generalized Number Field Sieve” (GNFS), which replaced the “Quadratic Sieve”(QS).

Have to assume computers will continue to get faster, and that better factoring algorithms may yet be found. Thus, we need to be careful in choosing a key size for RSA. For the near future, a key size in the range of 1024 to 2048 bits seems reasonable. In addition to specifying the size of n , a number of other constraints have been suggested by researchers. To avoid values of n that may be factored more easily, the algorithm's inventors suggest the following constraints on p and q :

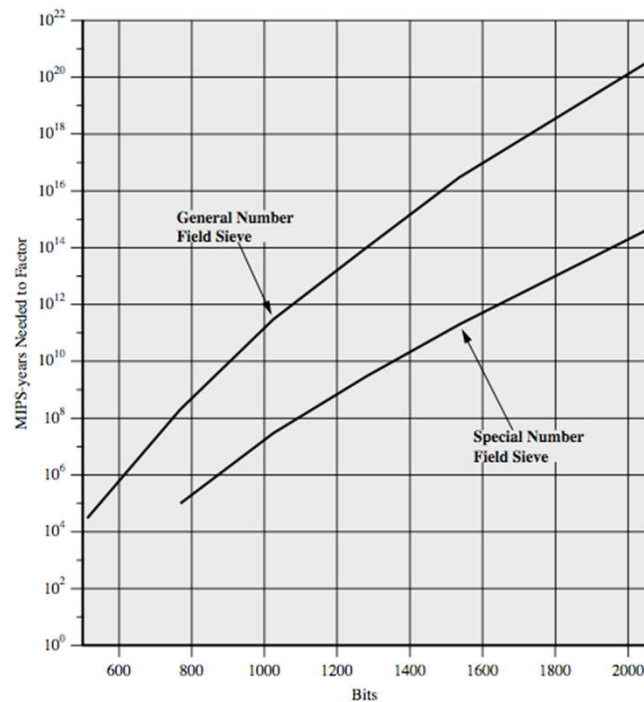
1. p and q should differ in length by only a few digits. Thus, for a 1024-bit key (309 decimal digits), both p and q should be on order of 10^{75} to 10^{100} .
2. Both $(p-1)$ and $(q-1)$ should contain a large prime factor
3. $\gcd(p-1, q-1)$ should be small.

Progress in Factoring

Number of Decimal Digits	Approximate Number of Bits	Date Achieved	MIPS-years	Algorithm
100	332	April 1991	7	quadratic sieve
110	365	April 1992	75	quadratic sieve
120	398	June 1993	830	quadratic sieve
129	428	April 1994	5000	quadratic sieve
130	431	April 1996	1000	generalized number field sieve
140	465	February 1999	2000	generalized number field sieve
155	512	August 1999	8000	generalized number field sieve
160	530	April 2003	—	Lattice sieve
174	576	December 2003	—	Lattice sieve
200	663	May 2005	—	Lattice sieve

Stallings Table 9.5 shows the progress in factoring to date. The level of effort is measured in MIPS-years: a million-instructions-per-second processor running for one year, which is about 3×10^{13} instructions executed. A 1 GHz Pentium is about a 250-MIPS machine.

Progress in Factoring



The threat to larger key sizes is twofold: the continuing increase in computing power, and the continuing refinement of factoring algorithms. We have seen that the move to a different algorithm resulted in a tremendous speedup. We can expect further refinements in the GNFS, and the use of an even better algorithm is also a possibility. In fact, a related algorithm, the special number field sieve (SNFS), can factor numbers with a specialized form considerably faster than the generalized number field sieve. Stallings Figure 9.9 compares the performance of the two algorithms. It is reasonable to expect a breakthrough that would enable a general factoring performance in about the same time as SNFS, or even better.

Timing Attacks

- developed by Paul Kocher in mid-1990's
- exploit timing variations in operations
 - eg. multiplying by small vs large number
 - or IF's varying which instructions executed
- infer operand size based on time taken
- RSA exploits time taken in exponentiation
- countermeasures
 - use constant exponentiation time
 - add random delays
 - blind values used in calculations

Have a radical new category of attacks developed by Paul Kocher in mid-1990's, based on observing how long it takes to compute the cryptographic operations. Timing attacks are applicable not just to RSA, but to other public-key cryptography systems. This attack is alarming for two reasons: It comes from a completely unexpected direction and it is a ciphertext-only attack. A timing attack is somewhat analogous to a burglar guessing the combination of a safe by observing how long it takes for someone to turn the dial from number to number. Although the timing attack is a serious threat, there are simple countermeasures that can be used, including using constant exponentiation time algorithms, adding random delays, or using blind values in calculations.

Chosen Ciphertext Attacks

RSA is vulnerable to a Chosen Ciphertext Attack (CCA)

attackers chooses ciphertexts & gets decrypted plaintext back

choose ciphertext to exploit properties of RSA to provide info to help cryptanalysis

can counter with random pad of plaintext or use Optimal Asymmetric Encryption Padding (OASP)

The RSA algorithm is vulnerable to a chosen ciphertext attack (CCA). CCA is defined as an attack in which adversary chooses a number of ciphertexts and is then given the corresponding plaintexts, decrypted with the target's private key. The adversary exploits properties of RSA and selects blocks of data that, when processed using the target's private key, yield information needed for cryptanalysis. Can counter simple attacks with random pad of plaintext. More sophisticated variants need to modify the plaintext using a procedure known as optimal asymmetric encryption padding (OAEP).

Key exchange

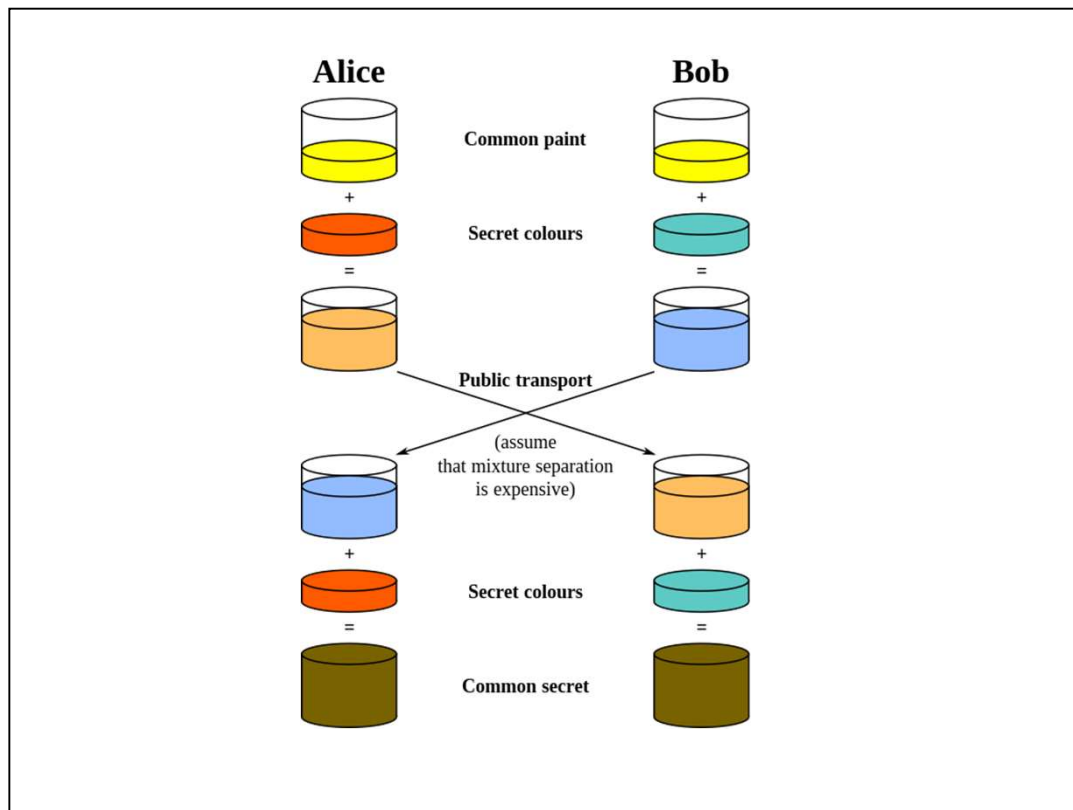
- The key exchange problem is how to exchange whatever keys or other information are needed so that no one else can obtain a copy.
- Key exchange is defined as any method in cryptography by which [cryptographic keys](#) are exchanged between two parties.

Deffie-Hellman

- **Diffie-Hellman** is a way of *generating* a shared secret between two people in such a way that the secret can't be seen by observing the communication. In other words:
you're creating a key together.
- This is particularly useful because you can use this technique to create an **encryption key** with someone, and then start encrypting your traffic with that key. And even if the traffic is recorded and later analyzed, there's absolutely no way to figure out what the key was, even though the exchanges that created it may have been visible.

Trapdoor function

- The way it works is reasonably simple. A lot of the math is the same as you see in public key crypto in that a [trapdoor function](#) is used.
- A **trapdoor function** is a function that is easy to compute in one direction, yet difficult to compute in the opposite direction (finding its inverse) without special information, called the "trapdoor".



Main idea

1. I come up with two prime numbers g and p and tell you what they are.
2. You then pick a secret number (a), but you don't tell anyone. Instead you compute $g^a \bmod p$ and send that result back to me. (We'll call that A since it came from a).
3. I do the same thing, but we'll call my secret number b and the computed number B . So I compute $g^b \bmod p$ and send you the result (called " B ")
4. Now, you take the number I sent you and do the exact same operation with *it*. So that's $B^a \bmod p$.
5. I do the same operation with the result you sent me, so: $A^b \bmod p$.

The "magic" here is that the answer I get at step 5 is *the same number* you got at step 4.

Shared Secret

- $(g^a \bmod p)^b \bmod p = g^{ab} \bmod p$
 $(g^b \bmod p)^a \bmod p = g^{ba} \bmod p$
- I never know what secret number you used to get to the result and you never know what number I used, but we still arrive at the same result.
- We can use that as our password for AES or Blowfish, or any other algorithm that uses shared secrets.

Example

Note: non-secret values in blue, and secret values in red

1. Alice and Bob agree to use a modulus $p = 23$ and base $g = 5$.
2. Alice chooses a secret integer $a = 6$, then sends Bob $A = g^a \bmod p$
 $A = 5^6 \bmod 23 = 8$
3. Bob chooses a secret integer $b = 15$, then sends Alice $B = g^b \bmod p$
 $B = 5^{15} \bmod 23 = 19$
4. Alice computes $s = B^a \bmod p$
 $s = 19^6 \bmod 23 = 2$
5. Bob computes $s = A^b \bmod p$
 $s = 8^{15} \bmod 23 = 2$
6. Alice and Bob now share a secret (the number 2).

Summary

- have considered:
 - principles of public-key cryptography
 - RSA algorithm, implementation, security

Chapter 9 summary.